



Instituto Politécnico da Guarda
Escola Superior de Tecnologia e Gestão

Caderno de exercícios
Programação em Python

**ESTE DOCUMENTO ESTÁ EM EDIÇÃO (VERSÃO: 2016-09-30
17:44)**

Setembro 2016

Curso: Engenharia Informática

Unidade Curricular: Algoritmos e Estruturas de Dados

Ano Letivo: 2016/2017

Docente: Paulo Jorge Costa Nunes

Coordenador da área disciplinar: Noel Lopes

Conteúdo

1	Introdução	9
2	Instalação e configuração do Python	11
3	A linguagem de programação Python	15
3.1	Funcionalidades	15
3.2	Características	15
3.3	Elementos de um programa Python	16
3.3.1	Nomes	16
3.3.2	Expressões e operadores	17
3.3.3	Funções de saída	17
3.3.4	Atribuição simples	18
3.3.5	Atribuição múltipla	19
3.3.6	Entrada e atribuição	19
3.3.7	Tipos de dados	19
3.4	Biblioteca de funções matemáticas	20
3.5	Conversão e arredondamento entre tipos de dados	21
3.6	Acumulador	23
3.7	Função range	24
3.8	Função list	25
3.9	Strings	26
3.9.1	Padrão travessia	27
3.10	Objectos	28
3.11	Estruturas de repetição	29
3.12	Módulos	29
3.13	A estrutura e sintaxe do Python	29
4	Cálculo de perímetros, áreas e volumes	31
4.1	Perímetro de um retângulo	31
4.1.1	Desenvolvimento do algoritmo	31
4.1.1.1	Modelo	31
4.1.1.2	Esboço	31
4.1.1.3	Algoritmo	32
4.1.2	Programa	33
4.1.2.1	Caso 1	33
4.2	Volume paralelepípedo	33
4.2.1	Desenvolvimento do algoritmo	33
4.2.1.1	Modelo	33
4.2.1.2	Esboço	34
4.2.1.3	Algoritmo	35
4.2.1.4	Programa	35

4.2.1.5	Caso 1	36
4.3	Proposta de exercícios	36
4.3.1	Perímetro de uma rotunda	36
4.3.2	Área de uma rotunda	36
4.3.3	Volume de um cilindro	36
5	Cálculo de médias e classificações qualitativas	37
5.1	Média de três números	37
5.2	Média de vários números	37
5.3	Classificação qualitativa	37
5.4	Qualidade do ar	37
6	Operações com data e horas	39
6.1	Contagem do número de dias de um dado dia da semana entre duas datas	39
6.2	Contagem do ocorrências entre duas datas com periodicidade	39
6.3	Número de dias até uma data	40
6.3.1	Geração de datas de eventos recorrentes	40
6.3.2	Eventos em tabela com nome do mês do dia da semana	40
6.3.2.1	Eventos em tabela numa dada língua	41
6.3.2.2	Eventos tabela HTML com interrupções	41
7	Operações com números	43
7.1	Decompor os dígitos de um número	43
8	Ciclos de repetição	45
8.1	Ciclo while	45

Lista de Figuras

2.1	Teste de instalação, janela shell interativa do Python.	11
2.2	Teste de instalação, janela shell interativa do Python.	12
2.3	Programa hello.py no editor Python.	12
2.4	Resultado do programa hello.py para o nome Ana.	13
3.1	Execução de um script Python em linha de comando MAC-OSX. Chamada e output.	16
3.2	Palavras reservados (31) do Python.	16
4.1	Modelo perímetro de um retângulo.	31
4.2	Caso 1: perímetro de um retângulo.	34
4.3	Modelo perímetro de um retângulo.	34
4.4	Caso 1: Programa Python volume de um paralelepípedo.	36
5.1	Classificação do Índice de Qualidade do Ar proposto para o ano 2013 (?g/m3) .	38
6.1	Interface eventos entre duas datas com periodicidade	39
6.2	Eventos entre duas datas.	40
6.3	Eventos entre duas datas numa tabela HTML em português.	41

Listings

2.1	Programa hello.py	12
3.1	Script Python Hello.	16
3.2	Exemplos da instrução print.	18
3.3	Exemplos de atribuição de valores a variáveis.	19
3.4	Entrada e atribuição de valores a variáveis.	19
3.5	Exemplos de atribuição de valores a variáveis.	20
3.6	Python Shell. Determinação to tipo de diversas variáveis.	20
3.7	Programa quadratic.py	21
3.8	Python Shell. Exemplos de conversão entre tipos.	22
3.9	Python Shell. Exemplos de conversão entre tipos reais.	23
3.10	Cálculo do valor de 6!	24
3.11	forma geral de um algoritmo acumulador	24
3.12	Programa para calcular 6! utilizando um acumulador.	24
3.13	Exemplo da utilização da função range().	24
3.14	Programa para calcular o fatorial de um número utilizando for e range().	25
3.15	Factorial do número 100.	25
3.16	Exemplo da utilização da função list().	25
3.17	Exemplo de strings e operações.	26
3.18	Travessia com while.	27
3.19	Travessia com for.	27
3.20	Saída dos programas tavessia com while e for	28
3.21	Travessia com for.	28
3.22	Programa futval.py	29
3.23	Exemplos de definição de constantes.	29
4.1	Esboço do algoritmo perímetro de um retângulo.	32
4.2	Algoritmo perímetro de um retângulo sem validação de dados de entrada.	32
4.3	Algoritmo perímetro de um retângulo.	33
4.4	Programa Python perímetro de um retângulo	33
4.5	Esboço do algoritmo o volume de um paralelepípedoo.	34
4.6	Algoritmo Programa Python volume de um paralelepípedo	35
4.7	Programa Python volume de um paralelepípedo	35
6.1	Exemplos de formatação de data e horas.	39
6.2		40
6.3	Seleção de zona de tempo (Europe/Lisboa) e localidade (Portugal: pt_BR).	40
8.1	Programa contador com ciclo while	45
8.2	Resultado do programa contador com ciclo while	45

Capítulo 1

Introdução

Com este caderno de exercícios pretende-se contribuir para o ensino da linguagem de programação Python na unidade curricular de Algoritmos e Estrutura de Dados do curso de Engenharia Informática.

Existem muitos recursos na Internet para desenvolvimento em Python, de seguida são apresentados alguns:

1. **Python** — Site oficial to Python <https://www.python.org>.
2. **Python 3.5.2 documentation** — <https://docs.python.org/3.5/>.
3. **The Python Package Index (PyPI)** — O PyPI é um repositório de software para a linguagem de programação Python. Atualmente (2016-09-12) com 88441 módulos desenvolvidos por terceiros <https://pypi.python.org/pypi>.
4. **Python** — <https://www.python.org>.
5. **Python** — <https://www.tutorialspoint.com/python/>.
6. **Book** — <http://www.openbookproject.net/thinkcs/python/english2e/>.
7. **How to Think Like a Computer Scientist: Learning with Python 2nd Edition documentation** — <http://www.openbookproject.net/thinkcs/python/english2e/#>.
8. **Book** — Python Programming: An Introduction to Computer Science by John M. Zelle, Ph.D <http://mcsp.wartburg.edu/zelle/python/>.
 - Programas Simples — Diretórios com os programas <http://mcsp.wartburg.edu/zelle/python/ppics3/code/>.
 - Código fonte dos programas para download — <http://mcsp.wartburg.edu/zelle/python/ppics3/code.zip>.
 - Slides — Slides Powerpoint para as aulas <http://mcsp.wartburg.edu/zelle/python/ppics3/slides/>.
9. **Book** - Practical Programming (2nd edition) An Introduction to Computer Science Using Python 3 by Paul Gries, Jennifer Campbell, Jason Montoyo — <https://pragprog.com/book/gwpy2/practical-programming>

- Programas Simples — Diretórios com os programas <http://mcsp.wartburg.edu/zelle/python/ppics3/code/>.
 - Código fonte dos programas para download — <http://mcsp.wartburg.edu/zelle/python/ppics3/code.zip>.
 - **Slides** — Slides Powerpoint para as aulas <http://mcsp.wartburg.edu/zelle/python/ppics3/slides/>.
10. **Book** — Invent Your Own Computer Games with Python, 2nd Edition 2nd Edition by Al Sweigart (Author) <http://inventwithpython.com/downloads/>.

Capítulo 2

Instalação e configuração do Python

No endereço <http://www.python.org/download/> estão disponíveis diversas versões do Python para diversos sistemas operativos/servidores web. Assim como as respectivas instruções de instalação e configuração do Python.

Os passos para instalar o Python no *Windows* são:

1. Visitar o endereço <http://www.python.org/download/> e descarregar (fazer o download) da versão mais atual do Python 3.
2. Double-click no ficheiro *.msi* descarregado e seguir as instruções. Todas as opções por omissão são adequadas para aprendizagem da linguagem por novos programadores.
3. Testar a instalação — Iniciar o *Python Shell* através do menu *Start* do Windows. Deverá aparecer uma janela semelhante à apresentada na figura 2.1 que tem o nome de *Python Shell*. O pronto (*prompt*) `> > >`, aguarda a escrita, a partir teclado, de operações do utilizador. As linhas `> > > 2 + 4`, `6` e `> > >` representam o resultado da operação de somar os números 2 e 4.

A figura 2.2 mostra a janela do editor IDLE (*Integrated DeveLopment Environment*) do Python semelhante à janela *Shell*. Esta janela apresenta uma barra de menus com diversas funcionalidades para editar, executar e fazer *debug* dos programas.

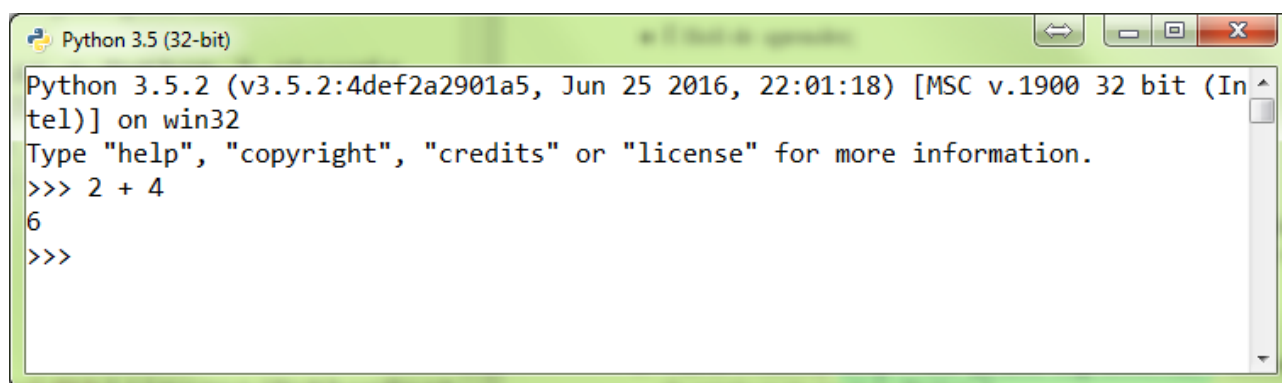


Figura 2.1: Teste de instalação, janela shell interativa do Python.

A listagem 2.1 mostra o código fonte de um programa escrito em Python. A linha 1 iniciada pelo símbolo `#` é um comentário em Python. As linhas de código 2 e 3 escrevem no ecrã e em linhas diferentes o texto contido entre os símbolos `'` (delimitadores de strings). A linha 4 lê a sequência de caracteres (nome) digitados pelo utilizador e armazena-os na variável `myName`. A

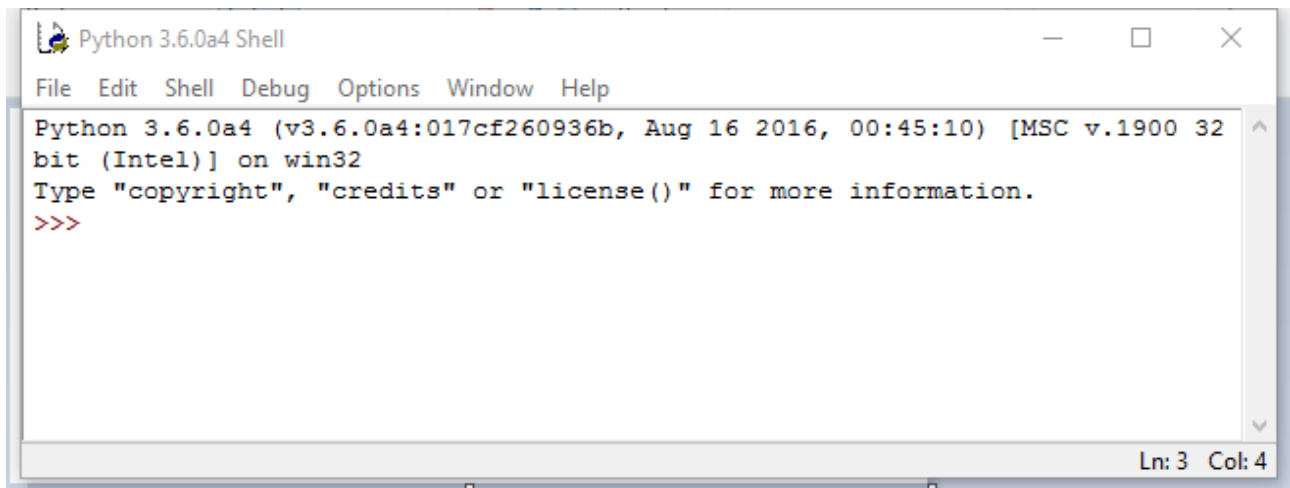
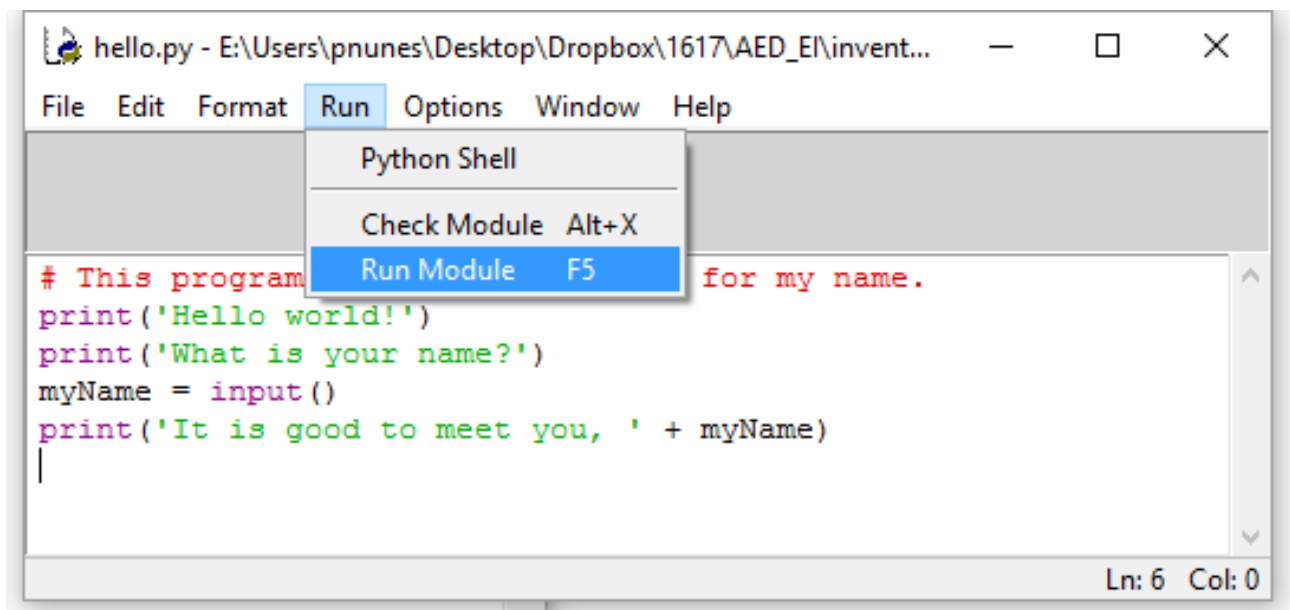


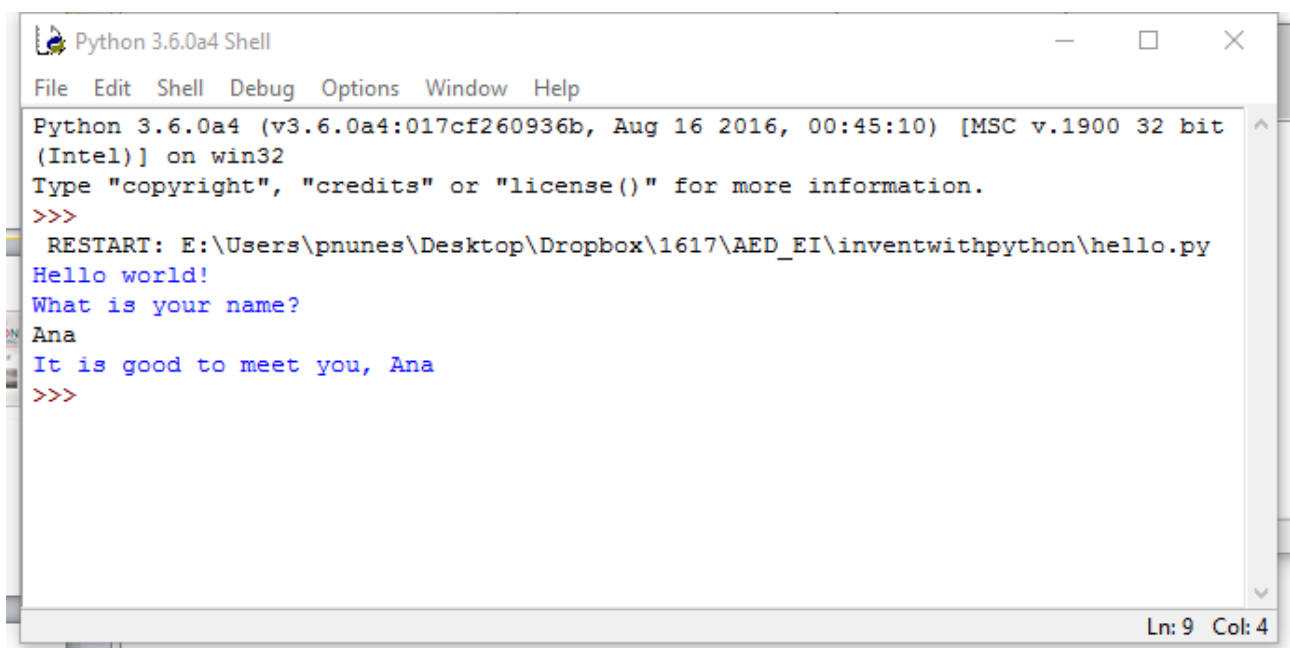
Figura 2.2: Teste de instalação, janela shell interativa do Python.

linha final escreve numa nova linha do ecrã o texto 'It is good to meet you, ', concatenado com o valor da referida variável (Ou seja o nome do utilizador).

As figuras 2.3 e 2.4 ilustram o programa `hello.py` no editor do Python e o output da sua execução com o Ana, respetivamente. A execução de programas pode ser efetuada através da opção *Run Module* do menu *Run*, como ilustra a 2.3.

```
1 # This program says hello and asks for my name.  
2 print('Hello world!')  
3 print('What is your name?')  
4 myName = input()  
5 print('It is good to meet you, ' + myName)
```

Listing 2.1: Programa `hello.py`Figura 2.3: Programa `hello.py` no editor Python.



The image shows a screenshot of a Windows command prompt window titled "Python 3.6.0a4 Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area displays the following content:

```
Python 3.6.0a4 (v3.6.0a4:017cf260936b, Aug 16 2016, 00:45:10) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: E:\Users\pnunes\Desktop\Dropbox\1617\AED_EI\inventwithpython\hello.py
Hello world!
What is your name?
Ana
It is good to meet you, Ana
>>>
```

The status bar at the bottom right of the window indicates "Ln: 9 Col: 4".

Figura 2.4: Resultado do programa hello.py para o nome Ana.

Capítulo 3

A linguagem de programação Python

3.1 Funcionalidades

O Python é uma linguagem extremamente modular ideal para ser instalado em servidores web na maioria dos sistemas operativos. Tem um conjunto enorme de funcionalidades, como por exemplo:

1. Desenvolvimento para a Web e Internet — Django, Pyramid, django CMS, HTML, XML, E-mail, FTP, IMAP, socket interface, HTTP client, etc.
2. Computação científica e numérica — SciPy, Pandas, IPython.
3. Acesso a bases de dados
4. Desenvolvimento de GUIs (Graphical User Interfaces) para Desktop — wxWidgets, Kivy, Qt via pyqt or pyside.
5. Educação — Python é uma linguagem excelente para ensinar programação, tanto a nível introdutório e em cursos mais avançados.
6. Programação com redes
7. Software & Game Development

3.2 Características

Algumas das principais características da linguagem de programação Python são listadas de seguida:

- O Python é potente ... e rápida;
- Pode ser executadas em qualquer sistema (computador/sistema operativo).
- É fácil de aprender;
- É open source.

3.3 Elementos de um programa Python

O Python pode ser executado na linha de comando de qualquer computador que tenha o Python instalado. Na listagem 3.1 é apresentado um script Python que escreve a mensagem "Hello from Python" utilizando o construtor de linguagem **echo**.

A figura 3.1 mostra um terminal com a execução e output do script **firstPythonScript.Python** em linha de comando através do comando bash **Python firstPythonScript.Python**

```
1 <?Python
2 echo "Hello from Python.";
```

Listing 3.1: Script Python Hello.

Figura 3.1: Execução de um script Python em linha de comando MAC-OSX. Chamada e output.

3.3.1 Nomes

Os nomes são dados a variáveis e módulos (secção 3.12). As linguagens de programação permitem manipular (atribuir e ler) variáveis que é uma das mais poderosas funcionalidades das linguagens de programação.

Os nomes das variáveis e módulos são chamados de identificadores. Cada identificador deve começar com uma letra ou um *underscore* (`_`), seguido de qualquer sequência de letras, dígitos ou *underscores*. O seu comprimento é arbitrário e não podem ser usados os nomes apresentados na 3.2 porque são palavras reservados (ou *keywords*) do Python. Isso significa que eles não estão disponíveis para ser usados como um nome para uma variável/módulo no programa.

Os identificadores são *case sensitive*. Ou seja, o identificador **preco** é diferente do identificador **Preco**.

and	as	assert	break	class	continue
def	del	elif	else	except	exec
finally	for	from	global	if	import
in	is	lambda	not	or	pass
print	raise	return	try	while	with
yield					

Figura 3.2: Palavras reservados (31) do Python.

Exemplos de identificadores válidos e diferentes:

- mensagem
- n
- pi
- numero_filhos

- X
- Celsius
- celsius
- cElsius

Exemplos de identificadores inválidos:

- and — Palavra reservada.
- 2peso — Não pode começar por um número.

3.3.2 Expressões e operadores

Uma expressão pode ser:

1. Um fragmento de código que produz ou calcula novos valores de dados.
2. Identificadores simples também podem ser expressões.
3. Literais — usados para representar um valor específico, por exemplo 3.9, 1, 1.0.
4. Dados textuais (strings) — Exemplo: "Olá".

As expressões simples podem ser combinadas utilizando operadores aritméticos (tabela 3.1), lógicos (tabela 3.2) e de manipulação de bits (tabela 3.3).

Tabela 3.1: Operadores aritméticos

Operador	Nome	Exemplo
+	adição	$2 + 3 = 5$
-	subtração	$2 - 3 = -1$
*	multiplicação	$2 * 3 = 6$
//	divisão inteira (divisão inteira trunca o resultado)	$5 / 2 = 2$
/	divisão real	$5 // 2 = 2.5$
%	módulo (resto)	$5 \% 4 = 1$
**	exponenciação	$2.0 \hat{=} 3.0 = 8$
/	raiz quadrada	$ / 25.0 = 5$
/	raiz cúbica	$ / 27.0 = 3$
!	fatorial	$5 ! = 120$
!!	fatorial (operador de prefixo)	$!! 5 = 120$

3.3.3 Funções de saída

As funções de saída (output statements) permitem apresentar os resultados de programa. A função de saída `print()` permite imprimir qualquer número de expressões na mesma linha e tem a seguinte sintaxe:

```
print(<expr>, <expr>, ?, <expr>)
```

Instruções de saída `print(...)` sucessivas imprimem em linhas separadas. A instrução `print()` sem argumentos imprime uma linha em branco. A listagem 3.2 mostra a instrução `print()` com diferentes argumentos e tipos de dados.

Tabela 3.2: Operadores lógicos

Operador	Nome
and	E lógico
or	OU lógico
not	Negação
<	Menos de
>	Maior que
<=	Menor ou igual a
>=	Maior ou igual a
==	Igual a
!=	Não igual a

Tabela 3.3: Operadores para manipulação de bits

Operador	Nome	Exemplo
&	AND bit a bit	91 & 15 = 11
	OR bit a bit	32 3 = 35
^	XOR bit a bit	5 ^ 3 = 6
	NOT bit a bit	1 = -2
«	deslocamento à esquerda bit a bit	1 « 4 = 16
»	deslocamento à direita bit a bit	8 » 2 = 2

```

1 print(3+4)
2 print(3, 4, 3+4)
3 print()
4 print(3, 4, end=" "),
5 print(3 + 4)
6 print("The answer is", 3+4)

```

Listing 3.2: Exemplos da instrução print.

Tabela 3.4: Saídas dos exemplos da listagem 3.2.

Instrução	Saída
>>> print(3+4)	7
>>> print(3, 4, 3+4)	3 4 7
>>> print()	
>>> print(3 + 4)	7
>>> print("The answer is", 3+4)	The answer is 7

3.3.4 Atribuição simples

Para atribuir valores a variáveis é utilizado o símbolo igual (=). A sintaxe é a seguinte:

<Variável> = <expr>

Onde **Variável** é um identificador e **expr** é uma expressão. Esta é avaliada para produzir um valor que é então associado com a variável.

```
1 x = 3.9 * x * (1-x)
2 fahrenheit = 9/5 * celsius + 32
3 x = 5
4 peso = 67.5
5 nome = "Ana"
6 curso = 'EI'
```

Listing 3.3: Exemplos de atribuição de valores a variáveis.

3.3.5 Atribuição múltipla

Em Python podem ser avaliadas várias expressões e atribuídos a vários variáveis ao mesmo tempo. A sintaxe é a seguinte:

`<Var>, <var>, ... = <expr>, <expr>, ...`

O exemplo: `x, y = x, y`, ao contrário de outras linguagens de programação (C, C++, Java, PHP, JavaScript) permite trocar o valor de duas variáveis sem a necessidade de usar uma variável auxiliar.

3.3.6 Entrada e atribuição

A função de entrada `input()` permite fazer a entrada de dados do utilizador (teclado). A sintaxe da função para leitura de número é a seguinte:

`<Variável> = eval (input (<prompt>))`

A função `eval()` avalia a cadeia de caracteres digitados pelo utilizador e transforma-o num valor (número em Python). A expressão `prompt` é uma cadeia de caracteres que representa uma mensagem a apresentar ao utilizador.

Para entrada de strings a sintaxe é a seguinte:

`<Variável> = input (<prompt>)`

Na listagem 3.4 são apresentados vários exemplo de entrada e atribuição de valores a variáveis de diversos tipos (números inteiros, número e reais cadeia de texto).

```
1 nome = input("Nome?")
2 idade = eval(input("Idade?"))
3 peso = eval(input("Peso (Kg)?"))
```

Listing 3.4: Entrada e atribuição de valores a variáveis.

3.3.7 Tipos de dados

No Python não é necessário declarar as variáveis. O tipo da variável depende do valor que armazena. O tipo da variável pode mudar várias vezes durante a execução do programa. Os tipos de variáveis básicos em Python são número inteiros, números reais e strings. A listagem 3.8 apresenta vários exemplos.

```
1  vencimento_base = 3437.34
2  pi = 3.14159
3  raiz_de2 = 1.4142124209
4  e = 2.71828
5  polegada = 2.54
6  grau_celcius = 32.0
7  velocidade_luz = 299792458
8  velocidade_som = 343.4
9  nome = "Carlos"
10 marca = "BMW"
11 marca = 1
```

Listing 3.5: Exemplos de atribuição de valores a variáveis.

O tipo de dados de uma variável determina quais os valores que ele pode ter e quais as operações que podem ser executadas nela. Em Python existe uma função especial para que permite obter o tipo de qualquer variável.

```
1 >>> type(3)
2 <class 'int'>
3 >>> type(3.1)
4 <class 'float'>
5 >>> nome = "Ana"
6 >>> type(nome)
7 <class 'str'>
8 >>>
```

Listing 3.6: Python Shell. Determinação do tipo de diversas variáveis.

3.4 Biblioteca de funções matemáticas

Uma biblioteca é um módulo (secção 3.12) com algumas definições e/ou funções de utilização geral. Ou seja, comum a vários programas. Na tabela 3.5 são mostradas algumas funções matemáticas da biblioteca *math* do Python.

Para usar as funções definidas numa biblioteca, é necessário incluí-la no programa utilizando a palavra reservada *import* do Python com a seguinte sintaxe:

```
import <nome_biblioteca>
```

Para importar a biblioteca de matemática *math* tem de ser incluída a seguinte linha de código no início do programa:

```
import math
```

A listagem 3.7 ilustra a utilização da função matemática raiz quadrada, *sqrt()*, para calcular as soluções da equação do 2º grau. Note-se que a referida função, na linha 14, está precedida do nome da biblioteca *math*.

Tabela 3.5: Funções matemáticas no Python.

Função/definições	Descrição	Exemplo/valor
pi	Uma aproximação do pi	3.141592653589793
e	Uma aproximação do e	2.718281828459045
abs()	valor absoluto	abs(-5.0) = 5
sqrt(x)	A raiz quadrada de x	
sin(x)	O seno de x	
cos(x)	O co-seno de x	
tan(x)	A tangente de x	
asin(x)	O inverso do seno x	
acos(x)	O inverso do cosseno x	
atan(x)	O inverso da tangente x	
log(x)	The natural (base e) logarithm of x	
log10(x)	The common (base 10) logarithm of x	
exp(x)	The exponential of x	
ceil(x)	The smallest integer not less than x	
floor(x)	The largest integer not greater than x	

```

1 # quadratic.py
2 #   A program that computes the real roots of a quadratic equation.
3 #   Illustrates use of the math library.
4 #   Note: This program crashes if the equation has no real roots.
5
6 import math # Makes the math library available.
7
8 def main():
9     print("This program finds the real solutions to a quadratic")
10    print()
11
12    a, b, c = eval(input("Please enter the coefficients (a, b, c): "))
13
14    discRoot = math.sqrt(b * b - 4 * a * c)
15    root1 = (-b + discRoot) / (2 * a)
16    root2 = (-b - discRoot) / (2 * a)
17
18    print()
19    print("The solutions are:", root1, root2 )

```

Listing 3.7: Programa quadratic.py

3.5 Conversão e arredondamento entre tipos de dados

Em expressões que envolvem números inteiro (*int*) e reais (*float*) o Python converte os números inteiros para números reais. Na conversão de um número real para um inteiro perde-se informação. Pois a conversão consiste em considerar apenas a parte inteiro do número e descartar a parte fracionário do número. Exemplos:

```
1 >>> 5 * 2
2 10
3 >>> 5.0 * 2
4 10.0
5 >>> 5 / 3
6 1.6666666666666667
7 >>> int(2.6)
8 2
9 >>> float(2)
10 2.0
11 >>> int("32")
12 32
13 >>> float("32")
14 32.0
```

Listing 3.8: Python Shell. Exemplos de conversão entre tipos.

O arredondamento de um número permite controlar a quantidade informação que se perde. A função `round()` permite arredondar um número real e tem a seguinte:

```
round( x [, n] )
```

Assim, podemos converter um número real (`x`) para outro número real com menos casas decimais (`n`). Para isso basta fornecer um segundo parâmetro que especifica o número de dígitos depois do ponto decimal (ver 3.9). A listagem 3.9 apresenta alguns exemplos com as funções `floor()`, `ceil()` e `round()`.

```

1 def main():
2
3     import math
4
5     print ("round(9.45, 2) : ", round(9.45, 2))
6     print ("round(9.45, 1) : ", round(9.45, 1))
7     print ("round(9.45, 0) : ", round(9.45, 0))
8
9     print ("math.ceil(9.45) : ", math.ceil(9.45))
10    print ("math.floor(9.45) : ", math.floor(9.45))
11    print ("math.ceil(9.65) : ", math.ceil(9.65))
12    print ("math.floor(9.65) : ", math.floor(9.65))
13
14    print ("math.ceil(-9.45) : ", math.ceil(-9.45))
15    print ("math.floor(-9.45) : ", math.floor(-9.45))
16    print ("math.ceil(-9.65) : ", math.ceil(-9.65))
17    print ("math.floor(-9.65) : ", math.floor(-9.65))
18
19    # https://www.tutorialspoint.com/python/
20    print ("round(80.23456, 2) : ", round(80.23456, 2))
21    print ("round(80.235, 2) : ", round(80.235, 2))
22    print ("round(80.2351, 2) : ", round(80.2351, 2))
23    print ("round(100.000056, 3) : ", round(100.000056, 3))
24    print ("round(-100.000056, 3) : ", round(-100.000056, 3))
25
26    main()
27
28    round(9.45, 2) :    9.45
29    round(9.45, 1) :    9.4
30    round(9.45, 0) :    9.0
31    math.ceil(9.45) :    10
32    math.floor(9.45) :    9
33    math.ceil(9.65) :    10
34    math.floor(9.65) :    9
35    math.ceil(-9.45) :    -9
36    math.floor(-9.45) :    -10
37    math.ceil(-9.65) :    -9
38    math.floor(-9.65) :    -10
39    round(80.23456, 2) :    80.23
40    round(80.235, 2) :    80.23
41    round(80.2351, 2) :    80.24
42    round(100.000056, 3) :    100.0
43    round(-100.000056, 3) :    -100.0
44

```

Listing 3.9: Python Shell. Exemplos de conversão entre tipos reais.

3.6 Acumulador

A listagem 3.10 ilustra o cálculo do valor do fatorial do número 6 ($6 \times 5 \times 4 \times 3 \times 2 \times 1 \times$) utilizando um acumulador. Ou seja, o valor obtido no passo anterior é reutilizado no passo corrente. Este algoritmo é conhecido como um acumulador, porque estamos construindo para cima ou para acumular a resposta em uma variável, conhecido como a variável acumulador.

```

1 Como calculamos 6 !?
2 6 * 5 = 30
3 Tome esse 30 e 30 * 4 = 120
4 Tome esse 120 e 120 * 3 = 360
5 Tome esse 360 e 360 * 2 = 720
6 Tome esse 720 e 720 * 1 = 720

```

Listing 3.10: Cálculo do valor de 6!

A forma geral de um algoritmo acumulador é a seguinte:

```

1 Inicializar a variável acumulador
2 Iterar até o resultado final ser alcançado
3   Atualizar o valor da variável acumulador

```

Listing 3.11: forma geral de um algoritmo acumulador

A forma geral de um algoritmo acumulador é a seguinte:

```

1 fact = 1
2 for x in [6, 5, 4, 3, 2]:
3     fact = fact * x
4 print(fact)

```

Listing 3.12: Programa para calcular 6! utilizando um acumulador.

3.7 Função range

A função `range()` permite gerar uma sequência de números. A sua sintaxe tem as seguintes possibilidades:

`range(<número_inteiro>)`

`range(<número_inicial>, <número_final>)`

`range(<número_inicial>, <número_final>, <incremento>)`

A listagem 1 apresenta exemplos para as diversas possibilidades de reutilização da função `range()`.

```

1 >>> for x in range(5): print(x)
2 0
3 1
4 2
5 3
6 4
7 >>> for x in range(2, 6): print(x)
8 2
9 3
10 4
11 5
12 >>> for x in range(5, 10, 2): print(x)
13 5
14 7
15 9
16 >>> for x in range(15, 9, -3): print(x)
17 15
18 12

```

Listing 3.13: Exemplo da utilização da função `range()`.

3.9 Strings

Uma string é uma sequência de caracteres entre aspas (") ou apóstrofo ('). A listagem 3.20 apresenta exemplos de strings e operações com elas. A tabela 3.6 apresenta uma string que representa um nome e a posição de cada letra do referido nome.

No endereço <http://www.openbookproject.net/thinkcs/python/english2e/ch07.html> estão disponibilizados muito exemplos que operação com strings.

Tabela 3.6: Letras de uma string e respetiva posição.

Letra	A	n	a		M	a	r	i	a		G	o	m	e	s
Posição	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

```

1 >>> nome = "Ana"
2 >>> apelido = 'Gomes'
3 >>> print(nome, apelido)
4 Ana Gomes
5 >>> nome_completo = nome + ' ' + apelido
6 >>> print(nome_completo)
7 Ana Gomes
8 >>>
9 >>> nome = 'Ana Maria Gomes'
10 >>> nome[0]
11 'A'
12 >>> print(nome[0], nome[4], nome[10])
13 A M G
14 >>> nome[-1]
15 's'
16 >>> nome[-2]
17 'e'
18 >>> nome[0:4]
19 'Ana '
20 >>> nome[0:3]
21 'Ana'
22 >>> nome[4:6]
23 'Ma'
24 >>> nome[4:9]
25 'Maria'
26 >>> nome[:9]
27 'Ana Maria'
28 >>> nome[9:]
29 'Gomes'
30 >>> nome[10:]
31 'Gomes'
32 >>> 3 * 'A' + 2 * 'B'
33 'AAABB'
34 >>> len(nome)
35 15
36

```

Listing 3.17: Exemplo de strings e operações.

O processamento de texto é uma das funções mais utilizada em computadores. A manipulação de texto envolve muitas operações sendo as mais comuns procurar texto e procurar e substituir texto. Outras operação envolvem a manipulação de texto. Por exemplo retirar o nome e o apelido de uma string que contem o nome completo de uma pessoa: Exemplo: dado

o nome o completo: "Ana Pereira Gomes", obter o nome: "Ana" e o apelido; "Gomes". Na tabela 3.7 são apresentados alguns operadores para strings incluindo exemplos.

Podemos ter acesso aos caracteres individuais em uma string através de indexação `[ ]`. As posições em uma string são numeradas a partir da esquerda, começando com 0. A forma geral é `<string> [<expr>]`, onde o valor de `expr` determina qual caractere é selecionado a partir da cadeia.

Tabela 3.7: Operadores para manipulação de strings

Operador	Significado
<code>+</code>	Concatenação
<code>*</code>	Repetição
<code><string>[]</code>	indexação
<code><string>[<start>:<end>]</code>	Sub-strings, não inclui o caractere na posição <code><end></code>
<code>len(<string>)</code>	Comprimento
<code>for <var> in <string></code>	Iteração através de personagens

3.9.1 Padrão travessia

Muitos cálculos em computador envolvem o processamento de uma sequência de caracteres, sendo processado um de cada vez. Muitas vezes, os algoritmos desenvolvidos para este tipo de processamento começam por selecionar o primeiro elemento, fazem um ou mais operações com ele e passam ao próximo elemento. Este processo repete-se até se atingir o último elemento. Este padrão de processamento é chamado de travessia.

Uma maneira de codificar uma travessia é com estruturas de repetição (`while`, `for`). Na listagem 3.18 apresentado um exemplo utilizando o `for` em na listagem 3.19 é apresentado o mesmo exemplo com o `while` [?].

```

1 nome = 'Ana Maria Gomes'
2 index = 0
3 while index < len(nome):
4     letter = nome[index]
5     print (letter)
6     index += 1

```

Listing 3.18: Travassia com while.

O ciclo `while` percorre a string e exibe cada letra numa linha. A condição de ciclo é o `index < len (nome)`, de modo que quando o `index` é igual ao comprimento da string, a condição é falsa, e o corpo do ciclo não é executado. O último caractere acedido é aquele com o índice `len (nome) -1`, que é o último caractere na string.

Usando um índice para percorrer um conjunto de valores é tão comum que Python oferece uma sintaxe alternativa simplificada, apresentada na listagem 3.19. A variável `c` (na linha 2) vai assumir cada uma dos caracteres da variável `nome` que é imprimido com a instrução `print(c)` (linha 3).

```

1 nome = 'Ana Maria Gomes'
2 for c in nome:
3     print (c)

```

Listing 3.19: Travessia com for.

```

1  A
2  n
3  a
4
5  M
6  a
7  r
8  i
9  a
10
11 G
12 o
13 m
14 e
15 s

```

Listing 3.20: Saída dos programas travessia com while e for .

A listagem 3.21 apresenta um programa que dado o número do mês apresenta as iniciais em inglês do respectivo mês. A variável **months** contém as iniciais (3 letras) de cada mês umas a seguir às outras começando pelo mês de janeiro e terminando no mês de dezembro. A tabela 3.8 apresenta a estrutura da variável **months**. Assim, as iniciais do mês de janeiro começam na posição 0, as do mês de fevereiro na posição 3, as do mês de março na posição 6, etc. Ou seja, as iniciais de cada mês estão a uma distância de 3 das do mês anterior.

Tabela 3.8: Iniciais dos meses do ano numa string.

Letra	J	a	n	F	e	b	M	a	r	A	p	r	M	a	y	J	u	n	J	u	l	A	u	g	S	e	
Posição	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	

```

1 # month.py
2 # A program to print the abbreviation of a month, given its
  # number
3
4 def main():
5
6     # months is used as a lookup table
7     months = "JanFebMarAprMayJunJulAugSepOctNovDec"
8
9     n = int(input("Enter a month number (1-12): "))
10
11     # compute starting position of month n in months
12     pos = (n-1) * 3
13
14     # Grab the appropriate slice from months
15     monthAbbrev = months[pos:pos+3]
16
17     # print the result
18     print("The month abbreviation is", monthAbbrev + ".")

```

Listing 3.21: Travessia com for.

3.10 Objectos

Objects are Python's abstraction for data. All data in a Python program is represented by objects or by relations between objects. (In a sense, and in conformance to Von Neumann's model of a "stored program computer," code is also represented by objects.)

3.11 Estruturas de repetição

A listagem 3.22 representa um programa para calcular o valor final de um investimento, dado o valor inicial, a taxa de juro e o número de anos.

```
1 # futval.py
2 #     A program to compute the value of an investment
3 #     carried n years into the future
4
5 def main():
6     print("This program calculates the future value of a 10-year
7         investment.")
8
9     principal = eval(input("Enter the initial principal: "))
10    apr = eval(input("Enter the annual interest rate (0-1): "))
11    n = eval(input("Enter the number of years: "))
12
13    for i in range(n):
14        principal = principal * (1 + apr)
15
16    print ("The value in ", n, " years is: ", principal)
17
18 main()
```

Listing 3.22: Programa futval.py

3.12 Módulos

```
1      sdsd
2      sds
3      dsd
4
5
```

Listing 3.23: Exemplos de definição de constantes.

3.13 A estrutura e sintaxe do Python

Capítulo 4

Cálculo de perímetros, áreas e volumes

4.1 Perímetro de um retângulo

Elabore um algoritmo/programa que permita calcular o perímetro de um retângulo.

4.1.1 Desenvolvimento do algoritmo

4.1.1.1 Modelo

A figura 4.1 ilustra o modelo para calcular o perímetro. A formula seguinte permite calcular o seu valor:

$$P = 2 \times L + 2 \times C$$

onde:

- L - Largura do retângulo ($L \in \mathbb{R}$)
- C - Comprimento do retângulo ($C \in \mathbb{R}$)
- P - Perímetro do retângulo ($P \in \mathbb{R}$)

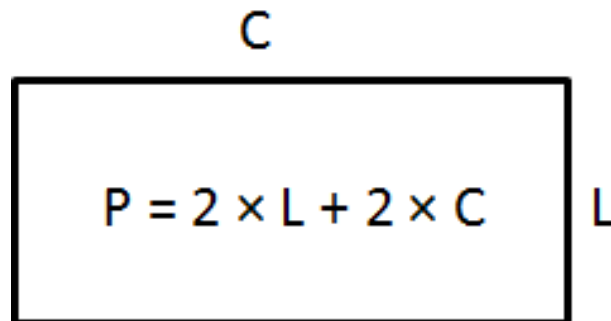


Figura 4.1: Modelo perímetro de um retângulo.

4.1.1.2 Esboço

```
1 LER C
2 LER L
3 P = 2 x C + 2 x L
4 ESCREVER P
5
```

Listing 4.1: Esboço do algoritmo perímetro de um retângulo.

4.1.1.3 Algoritmo

```
1 Algoritmo: algoritmoPerimetroRetangulo
2 Objetivo: Pertmite calcular o perímetro de um retângulo
3 Variáveis
4 Entrada:
5   L (Inteiro T2) - Largura (> 0, <= 99)
6   C (Inteiro T2) - Comprimento (> 0, <= 99)
7 Saída:
8   P (Inteiro T5) - Perímetro (> 0, <= 99999)
9 Data: 2016-9-26 13:46:11
10 Autor: Paulo Nunes
11 Versão: 1.0
12 Obs:
13 Início:
14   /* Entrada de dados (INPUT) */
15   ESCREVER "Largura?"
16   ESCREVER "Comprimento?"
17   /* Processamento (PROCESSING) */
18   P = 2 x L + e x C
19   /* Saída de resultados (OUTPUT) */
20   ESCREVER "Perímetro: ", P
21 Fim.
22
```

Listing 4.2: Algoritmo perímetro de um retângulo sem validação de dados de entrada.


```

1  Algoritmo: algoritmoPerimetroRetangulo
2  Objetivo: Pertmite calcular o perímetro de um retângulo
3  Variáveis
4  Entrada:
5      L (Inteiro T2) - Largura (> 0, <= 99)
6      C (Inteiro T2) - Comprimento (> 0, <= 99)
7  Saída:
8      P (Inteiro T5) - Perímetro (> 0, <= 99999)
9  Data: 2016-9-26 13:46:11
10 Autor: Paulo Nunes
11 Versão: 1.0
12 Obs:
13 Início:
14 /* Entrada de dados (INPUT) */
15 FAZER
16     ESCREVER "Largura?"
17     LER L
18 ATÉ ( (L > 0) E (L <= 99) )
19 FAZER
20     ESCREVER "Comprimento?"
21     LER C
22 ATÉ ( (C > 0) E (C <= 99) )
23 /* Processamento (PROCESSING) */
24 P = 2 x L + e x C
25 /* Saída de resultados (OUTPUT) */
26 ESCREVER "Perímetro: ", P
27 Fim.
28

```

Listing 4.3: Algoritmo perímetro de um retângulo.

4.1.2 Programa

```

1 def main():
2     L = eval(input("Largura do retângulo: "))
3     C = eval(input("Comprimento do retângulo: "))
4     P = 2 * L + 2 * C
5     print("Perímetro do retângulo é: ", P)
6
7 main()

```

Listing 4.4: Programa Python perímetro de um retângulo

4.1.2.1 Caso 1

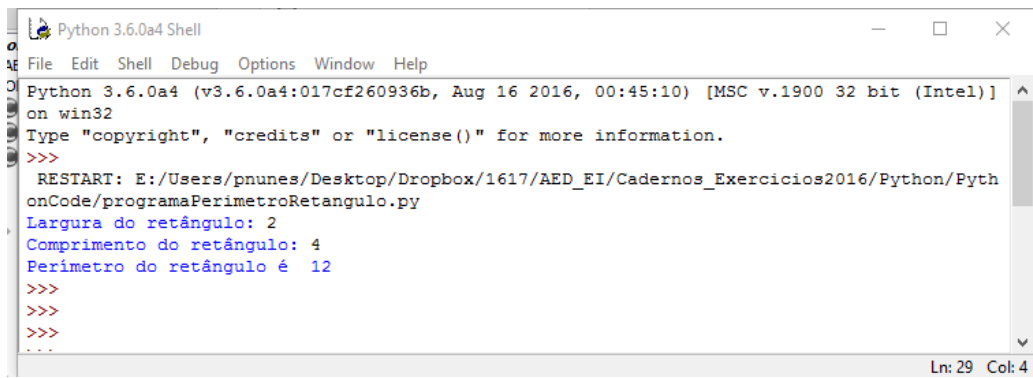
4.2 Volume paralelepípedo

Elabore um algoritmo/programa que permita calcular o volume de um paralelepípedo, de acordo com a figura seguinte (4.3).

4.2.1 Desenvolvimento do algoritmo

4.2.1.1 Modelo

A figura 4.3 ilustra o modelo para calcular o volume de um paralelepípedo. A formula seguinte permite calcular o seu valor:



```

Python 3.6.0a4 Shell
File Edit Shell Debug Options Window Help
Python 3.6.0a4 (v3.6.0a4:017cf260936b, Aug 16 2016, 00:45:10) [MSC v.1900 32 bit (Intel)]
on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: E:/Users/pnunes/Desktop/Dropbox/1617/AED_EI/Cadernos_Exercicios2016/Python/Pyth
onCode/programaPerimetroRetangulo.py
Largura do retângulo: 2
Comprimento do retângulo: 4
Perímetro do retângulo é 12
>>>
>>>
>>>
...
Ln: 29 Col: 4

```

Figura 4.2: Caso 1: perímetro de um retângulo.

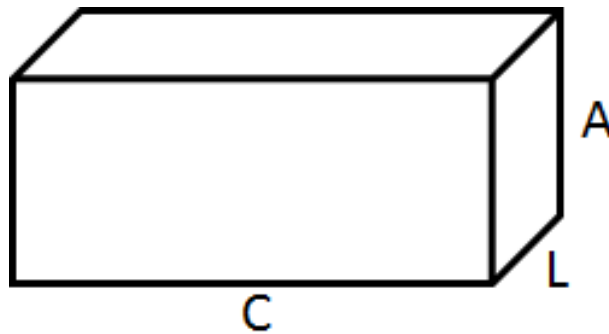


Figura 4.3: Modelo perímetro de um retângulo.

$$V = C \times L + \times C \times A$$

onde:

- L - Largura do paralelepípedo ($L \in \mathbb{R}$)
- C - Comprimento do paralelepípedo ($C \in \mathbb{R}$)
- A - Altura do paralelepípedo ($A \in \mathbb{R}$)
- V - Volume do paralelepípedo ($V \in \mathbb{R}$)

4.2.1.2 Esboço

```

1  LER C
2  LER L
3  LER A
4  V = C x L x A
5  ESCREVER V
6

```

Listing 4.5: Esboço do algoritmo o volume de um paralelepípedoo.

4.2.1.3 Algoritmo

```

1  Algoritmo: programaVolumeParalelepipedo
2  Objetivo:
3      Permite calcular o volume de um paralelepípedo
4  Variáveis
5  Entrada:
6      L (Real T6.3) - Largura do paralelepípedo (m) (> 0, <= 999.999)
7      C (Real T6.3) - Comprimento do paralelepípedo (m) (> 0, <= 999.999)
8      A (Inteiro T6.3) - Altura do paralelepípedo (m) (> 0, <= 999.999)
9  Saída:
10     V (Real T12.3) - Volume do paralelepípedo (m3) (> 0.0, <= 999999999.999)
11 Data: 2016-9-26 19:4:15
12 Autor: Paulo Nunes
13 Versão: 1.0
14 Obs:
15 Início:
16     /* Entrada de dados (INPUT) */
17     FAZER
18         ESCREVER "Largura do paralelepípedo (m)?"
19         LER L
20     ATÉ ( (L > 0) E (L <= 999.999) )
21     FAZER
22         ESCREVER "Comprimento do paralelepípedo (m)?"
23         LER C
24     ATÉ ( (C > 0) E (C <= 999.999) )
25     FAZER
26         ESCREVER "Altura do paralelepípedo (m)?"
27         LER A
28     ATÉ ( (A > 0) E (A <= 999.999) )
29     /* Processamento (PROCESSING) */
30     V = C * L * A
31     /* Saída de resultados (OUTPUT) */
32     ESCREVER "Volume do paralelepípedo: ", V, " m3"
33 Fim.
34

```

Listing 4.6: Algoritmo Programa Python volume de um paralelepípedo

4.2.1.4 Programa

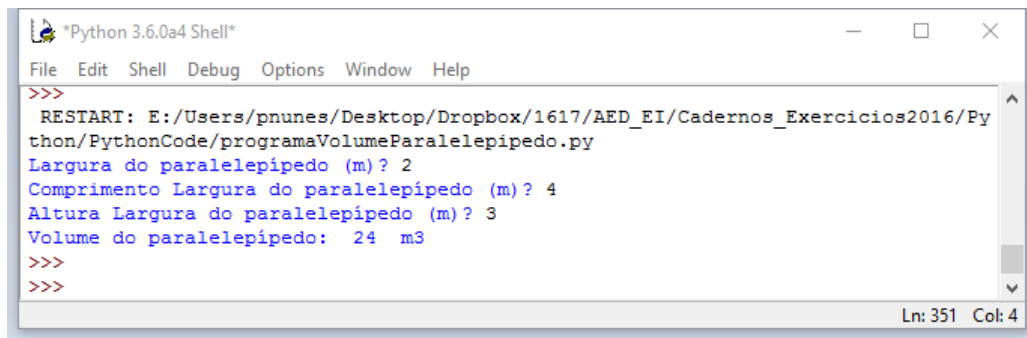
```

1  def programaVolumeParalelepipedo():
2      L = -1
3      while ((L <= 0.0) or (L >= 999.999)):
4          L = eval(input("Largura do paralelepípedo (m)? "))
5      C = -1
6      while ((C <= 0.0) or (C >= 999.999)):
7          C = eval(input("Comprimento Largura do paralelepípedo (m)? "))
8      A = -1
9      while ((A <= 0.0) or (A >= 999.999)):
10         A = eval(input("Altura Largura do paralelepípedo (m)? "))
11     V = C * L * A
12     print("Volume do paralelepípedo: ", V, " m3")
13
14 programaVolumeParalelepipedo()

```

Listing 4.7: Programa Python volume de um paralelepípedo

4.2.1.5 Caso 1



```
*Python 3.6.0a4 Shell*
File Edit Shell Debug Options Window Help
>>>
RESTART: E:/Users/pnunes/Desktop/Dropbox/1617/AED_EI/Cadernos_Exercicios2016/Py
thon/PythonCode/programaVolumeParalelepipedo.py
Largura do paralelepipedo (m)? 2
Comprimento Largura do paralelepipedo (m)? 4
Altura Largura do paralelepipedo (m)? 3
Volume do paralelepipedo: 24 m3
>>>
>>>
```

Figura 4.4: Caso 1: Programa Python volume de um paralelepípedo.

4.3 Proposta de exercícios

4.3.1 Perímetro de uma rotunda

Elabore um algoritmo para calcular o perímetro de uma rotunda. Assim como o número peças de cimento para circundar a rotunda.

4.3.2 Área de uma rotunda

Elabore um algoritmo para calcular a área de uma rotunda. Assim como o número de pés de flor para plantar na rotunda.

4.3.3 Volume de um cilindro

Elabore um algoritmo para calcular o volume de um cilindro.

Capítulo 5

Cálculo de médias e classificações qualitativas

5.1 Média de três números

Elabore um algoritmo para calcular a média de três número.

5.2 Média de vários números

Elabore um algoritmo para calcular a média das notas dos alunos de uma disciplina.

5.3 Classificação qualitativa

Elabore um algoritmo em fluxograma para atribuir uma classificação qualitativa de acordo com a seguinte tabela:

Tabela 5.1: Tabela de classificação

Nota	Classificação
0 a 9	Aprovado
10 a 20	Reprovado

5.4 Qualidade do ar

Elabore um algoritmo que permita determinar a qualidade do ar com base na concentração de CO.

Poluente em causa / Classificação	CO	
	Min	Máx
Mau	10000	-----
Fraco	8500	9999
Médio	7000	8499
Bom	5000	6999
Muito Bom	0	4999

Figura 5.1: Classificação do Índice de Qualidade do Ar proposto para o ano 2013 ($\mu\text{g}/\text{m}^3$)

Capítulo 6

Operações com data e horas

A listagem 6.1 mostra vários exemplos de formatação da data e hora obtidas do sistema operativo.

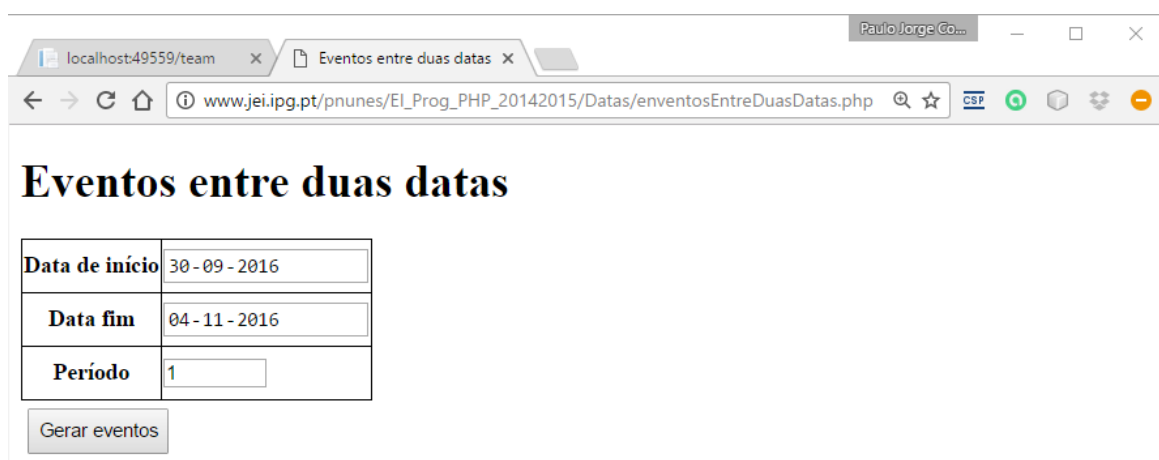
```
1 ...
```

Listing 6.1: Exemplos de formatação de data e horas.

6.1 Contagem do número de dias de um dado dia da semana entre duas datas

Elabore um algoritmo que permita calcular o número de dias de um dado dia da semana (seg, ter, ..., dom) entre duas datas. Exemplo:

6.2 Contagem do ocorrências entre duas datas com periodicidade



The screenshot shows a web browser window with the title 'Eventos entre duas datas'. The address bar shows the URL 'www.jei.ipg.pt/pnunes/EI_Prog_PHP_20142015/Datas/enventosEntreDuasDatas.php'. The main content area has the title 'Eventos entre duas datas' and a form with the following fields:

Data de início	30-09-2016
Data fim	04-11-2016
Período	1

Below the form is a button labeled 'Gerar eventos'.

Figura 6.1: Interface eventos entre duas datas com periodicidade

```

1 Eventos entre 2016-09-19 e 2016-12-19
2
3 Período 7
4
5 1 - 2016-09-19
6 2 - 2016-09-26
7 3 - 2016-10-03
8 4 - 2016-10-10
9 5 - 2016-10-17
10 6 - 2016-10-24
11 7 - 2016-10-31
12 8 - 2016-11-07
13 9 - 2016-11-14
14 10 - 2016-11-21
15 11 - 2016-11-28
16 12 - 2016-12-05
17 13 - 2016-12-12
18 14 - 2016-12-19

```

6.3 Número de dias até uma data

...

6.3.1 Geração de datas de eventos recorrentes

As agendas eletrônicas permitem gerar uma sequência de datas de eventos recorrentes. Com base na data do primeiro evento, o número de eventos e o período de tempo que cada evento se repete (semana, quinzena) geram a sequência de datas dos eventos. Neste exercício pretende-se gerar uma sequência de datas de eventos entre duas datas com uma dada periodicidade fornecidos pelo utilizador.

A figura 6.2 ilustra um exemplo de geração de eventos com um período de 7 dias.

(a) Input

(b) Output

Figura 6.2: Eventos entre duas datas.

```

1 ...

```

Listing 6.2: ...

6.3.2 Eventos em tabela com nome do mês do dia da semana

Altere o exemplo anterior de forma a apresentar o resultado numa tabela HTML com o aspeto da figura 6.3. Utilize o código da listagem 6.3 para seleccionar zona de tempo Europe/Lisbon e local *pt_BR*.

Listing 6.3: Seleção de zona de tempo (Europe/Lisboa) e localidade (Portugal: *pt_BR*).

A figura ?? ilustra um exemplo de geração de eventos com um período de 7 dias.

(a) Input

(b) Output

Figura 6.3: Eventos entre duas datas numa tabela HTML em português.

6.3.2.1 Eventos em tabela numa dada língua

Adicione uma caixa de seleção ao interface da figura ?? de modo permitir ao utilizador seleccionar a língua na qual deseja obter o output. Efetue as alterações necessárias no código que gera o output para produzir o output de acordo com a língua seleccionada.

6.3.2.2 Eventos tabela HTML com interrupções

Considere o exercício da secção 6.3.2.1. Adicione uma tabela com duas caixas de entradas de dados, data de início e data de fim de interrupção, um botão para dicionar linhas no fim da tabela e um botão para remover a última linha da tabela. A figura ?? ilustra o aspecto do interface pretendido.

A figura ?? mostra o output dos eventos com início a 25-02-2015 e fim a 19-05-201, uma periodicidade de 7 dias e com duas interrupções. A primeira, para o Carnaval, entre os dias 16-02-2015 a 17-02-2015 e a segunda, para a Páscoa, entre os dias 30-03-2015 a 06-04-2015.

Capítulo 7

Operações com números

7.1 Decompor os dígitos de um número

Elabore um programa para decompor os dígitos de um número.

Capítulo 8

Ciclos de repetição

8.1 Ciclo while

A while loop statement in Python programming language repeatedly executes a target statement as long as a given condition is true.

A sintaxe do ciclo while é:

```
while expression:  
    statement(s)
```

```
1 contador = 0  
2 while (contador <= 9):  
3     print ('0 contador Ã©:', contador)  
4     contador = contador + 1
```

Listing 8.1: Programa contador com ciclo while

```
1 0 contador é: 0  
2 0 contador é: 1  
3 0 contador é: 2  
4 0 contador é: 3  
5 0 contador é: 4  
6 0 contador é: 5  
7 0 contador é: 6  
8 0 contador é: 7  
9 0 contador é: 8  
10 0 contador é: 9  
11 >>>
```

Listing 8.2: Resultado do programa contador com ciclo while